

UT 01. Valores y variables

Índice

- 1.1 Ejecución de JavaScript
- 1.2 Tipos y el operador **typeof**
- 1.3 Comentarios
- 1.4 Declaraciones de variables
- 1.5 Identificadores
- 1.6 Números
- 1.7 Operadores aritméticos
- 1.8 Valores booleanos
- 1.9 **null** y **undefined**
- 1.10 Literales de cadena
- 1.11 Literales de plantilla
- 1.12 Objetos
- 1.13 Sintaxis de literales de objetos
- 1.14 Matrices
- 1.15 JSON
- 1.16 Desestructuración
- 1.17 Desestructuración avanzada

En este capítulo, aprenderás sobre los tipos de datos que puedes manipular en un programa JavaScript: números, cadenas y otros tipos primitivos, así como objetos y matrices. Verás cómo almacenar estos valores en variables, cómo convertir valores de un tipo a otro y cómo combinar valores con operadores.

Incluso los programadores JavaScript más entusiastas estarán de acuerdo en que algunas construcciones del lenguaje, pensadas para ayudar a escribir programas cortos, pueden dar lugar a resultados poco intuitivos y es mejor evitarlas. En este capítulo y en los siguientes, señalaré estos problemas y proporcionaré reglas sencillas para programar de forma segura.

1.1 Ejecución de JavaScript

Para ejecutar programas JavaScript mientras lees este libro, puedes utilizar varios métodos diferentes.

JavaScript fue diseñado originalmente para ejecutarse en un navegador. Puede incrustar JavaScript en un archivo HTML e invocar el método `window.alert` para mostrar valores. A modo de ejemplo, aquí tiene un archivo de este tipo:

```
<html>
  <head>
    <title>Mi primer programa JavaScript</title>
    <script type="text/javascript">
      let a = 6
      let b = 7
      window.alert(a * b)
    </script>
```

```
</head>
<body>
</body>
</html>
```

Simplemente abra el archivo en su navegador web favorito y el resultado se mostrará en un cuadro de diálogo (véase **la figura 1-1**).



Figura 1-1 Ejecución de código JavaScript en un navegador web

Puede escribir secuencias de instrucciones cortas en la consola que forma parte de las herramientas de desarrollo de su navegador. Busque el menú o el atajo de teclado para mostrar las herramientas de desarrollo (en muchos navegadores, es la tecla F12, o la combinación de teclas Ctrl+Alt+I, o, en Mac, la combinación de teclas Cmd+Alt+I). A continuación, seleccione la pestaña «Consola» y escriba su código JavaScript (véase **la figura 1-2**).

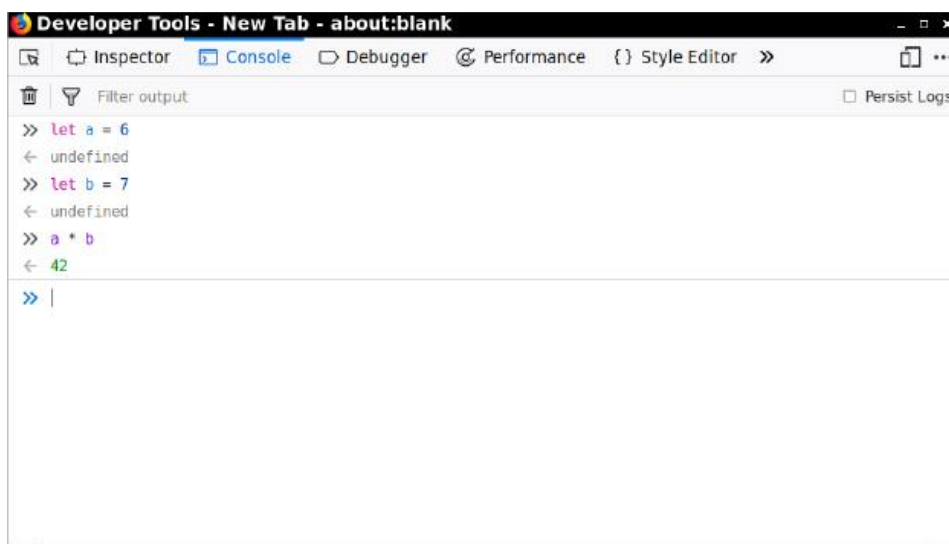


Figura 1-2. Ejecución de código JavaScript en la consola de herramientas de desarrollo

Un tercer método consiste en instalar Node.js desde <http://nodejs.org>. A continuación, abra un terminal y ejecute el programa **node**, que inicia un «bucle de lectura-evaluación-impresión» (REPL) de JavaScript. Escriba los comandos y vea sus resultados, como se muestra en **la Figura 1-3**.

A terminal window titled "Terminal - \$" showing the execution of JavaScript code in the Node.js REPL. The user enters the command "node" at the prompt, followed by three lines of code: "let a = 6", "let b = 7", and "a * b". The REPL outputs "undefined" for the first two lines and "42" for the third line. A red cursor is visible on the line "a * b".

```
Terminal - $
~$ node
> let a = 6
undefined
> let b = 7
undefined
> a * b
42
>
```

Figura 1-3 Ejecución de código JavaScript con el REPL de Node.js

Para secuencias de código más largas, coloque las instrucciones en un archivo y utilice el método `console.log` para generar la salida. Por ejemplo, puede colocar estas instrucciones en un archivo `first.js`:

```
let a = 6
let b = 7
console.log(a * b)
```

A continuación, ejecute el comando

```
node first.js
```

El resultado del comando `console.log` se mostrará en la terminal.

También puede utilizar un entorno de desarrollo como Visual Studio Code, Eclipse, Komodo o WebStorm. Estos entornos permiten editar y ejecutar código JavaScript, como se muestra en **la Figura 1-4**.

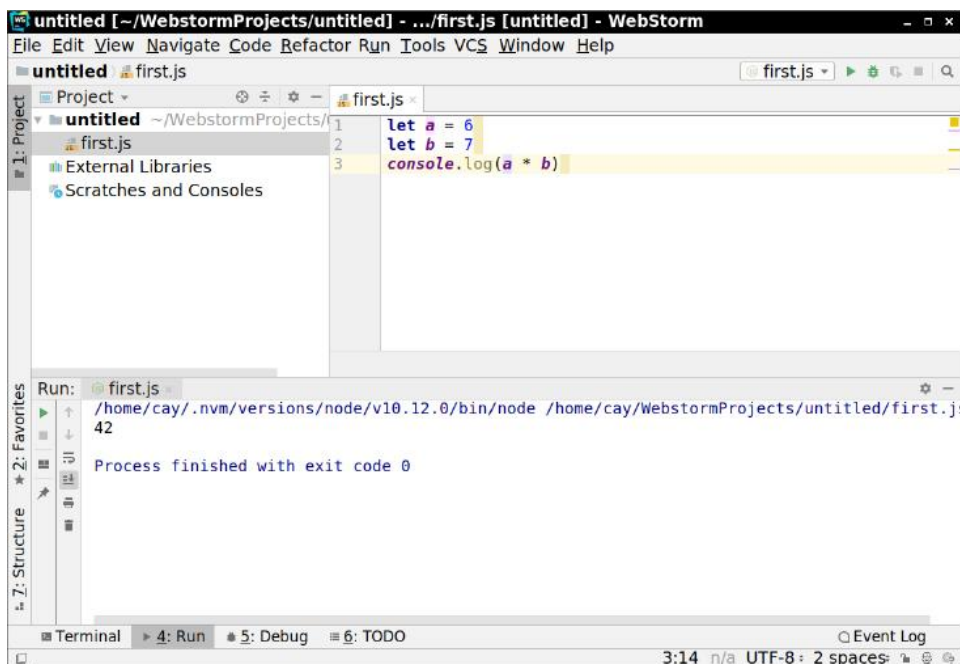


Figura 1-4 Ejecución de código JavaScript en un entorno de desarrollo

1.2 Tipos y el operador `typeof`

Todos los valores en JavaScript son de uno de los siguientes tipos:

- Un número
- Los valores booleanos *false* y *true*
- Los valores especiales *null* y *undefined*
- Una cadena
- Un símbolo
- Un objeto

Los tipos que no son objetos se denominan colectivamente *tipos primitivos*.

Dado un valor, puede encontrar su tipo con el operador `typeof`, que devuelve una cadena «number», «boolean», «undefined», «object», «string», «symbol» o una de otras pocas cadenas. Por ejemplo, `typeof 42` es la cadena «number».



Nota

Aunque el tipo `null` es distinto del tipo `object`, `typeof null` es la cadena «object». Se trata de una casualidad histórica.



Precaución

Al igual que en Java, puede construir objetos que envuelvan números, valores booleanos y cadenas. Por ejemplo, `typeof new Number(42)` y `typeof new String('Hello')` son «object». Sin embargo, en JavaScript no hay ninguna razón válida para construir este tipo de instancias envolventes. Dado que pueden ser motivo de confusión, las normas de codificación suelen prohibir su uso.

1.3 Comentarios

JavaScript tiene dos tipos de comentarios. Los comentarios de una sola línea comienzan con `//` y se extienden hasta el final de la línea

```
// como este
```

Los comentarios delimitados por `/*` y `*/` pueden abarcar varias líneas

```
/*
  como
  así
*/
```



Nota

A diferencia de Java, JavaScript no tiene una forma especial de comentarios de documentación. Sin embargo, existen herramientas de terceros, como JSDoc (<http://usejsdoc.org>), que ofrecen la misma funcionalidad.

1.4 Declaraciones de variables

Puedes almacenar un valor en una variable con la instrucción `let`:

```
let counter = 0
```

En JavaScript, las variables no tienen tipo. Se puede almacenar valores de cualquier tipo en cualquier variable. Por ejemplo, es válido sustituir el contenido de `counter` por una cadena:

```
counter = 'zero'
```

Sin embargo, es casi seguro que no sea una buena idea hacerlo. No obstante, hay situaciones en las que tener variables sin tipo facilita la escritura de código genérico que funciona con diferentes tipos.

Si no inicializas una variable, esta tiene el valor especial `undefined`:

```
let x // Declara x y lo establece como indefinido
```



Nota

Quizás hayas notado que las instrucciones anteriores no terminan con punto y coma. En JavaScript, al igual que en Python, no es necesario usar punto y coma al final de una línea. En Python, se considera «no pythónico» añadir puntos y comas innecesarios. Sin embargo, los programadores de JavaScript están divididos sobre esta cuestión.

Si nunca cambias el valor de una variable, debes declararla con una instrucción `const`:

```
const PI = 3.141592653589793
```

Si intentas modificar el valor contenido en una `const`, se produce un error en tiempo de ejecución.

Puedes declarar varias variables con una sola instrucción `const` o `let`:

```
const FREEZING = 0, BOILING = 100  
let x, y
```

Sin embargo, muchos programadores prefieren declarar cada variable con una instrucción independiente.



Precaución

Evite dos formas obsoletas de declarar variables, con la palabra clave `var` y sin ninguna palabra clave:

```
var counter = 0 // Obsoleto  
coutner = 1 // Observe el error ortográfico: ¡crea una nueva variable!
```

La declaración `var` tiene algunas deficiencias graves; hablaremos de ellas más tarde. El comportamiento de «crear en la primera asignación» es obviamente peligroso. Si escribes mal el nombre de una variable, se crea una nueva variable. Por esa razón, «crear en la primera asignación» es un error en *el modo estricto*, un modo que prohíbe ciertas construcciones obsoletas



Consejo

Hay dos reglas de oro que, si se siguen, eliminan la mayor parte de la confusión causada por las características «clásicas» de JavaScript. Las dos primeras reglas de oro son:

1. Declara las variables con `let` o `const`, no con `var`.
2. Utiliza el modo estricto.

1.5 Identificadores

El nombre de una variable debe seguir la sintaxis general de *los identificadores*. Un identificador se compone de letras Unicode, dígitos y los caracteres `_` y `$`. El primer carácter no puede ser un dígito. Los nombres con caracteres `$` se utilizan a veces en herramientas y bibliotecas. Algunos programadores utilizan identificadores que comienzan o terminan con guiones bajos para indicar características «privadas». Con

tus propios nombres, es mejor evitar tanto el \$ como el _ al principio o al final. Los caracteres _ internos están bien, pero muchos programadores de JavaScript prefieren el formato camelCase, en el que se utilizan letras mayúsculas para delimitar las palabras.

No se pueden utilizar las siguientes palabras clave como identificadores:

```
break case catch class const continue debugger default delete do
else enum export extends false finally for function if import in instanceof
new null return super switch this throw true try typeof var void while with
```

En modo estricto, estas palabras clave también están prohibidas:

```
implements interface let package protected private public static
```

Las siguientes palabras clave son adiciones más recientes al lenguaje; puede utilizarlas como identificadores para garantizar la compatibilidad con versiones anteriores, pero no debería hacerlo:

```
await as async from get of set target yield
```



Nota

Puede utilizar cualquier letra o dígito Unicode en los identificadores, como, por ejemplo:

```
const π = 3.141592653589793
```

Sin embargo, esto no es habitual, probablemente porque muchos programadores carecen de métodos de entrada para escribir dichos caracteres.

1.6 Números

JavaScript no tiene un tipo entero explícito. Todos los números son números de doble precisión en coma flotante. Por supuesto, se pueden utilizar valores enteros; simplemente no hay que preocuparse por la diferencia entre, por ejemplo, 1 y 1,0. ¿Qué hay del redondeo? Cualquier número entero entre `Number.MIN_SAFE_INTEGER` ($-2^{53} + 1$ o $-9.007.199.254.740.991$) y `Number.MAX_SAFE_INTEGER` ($2^{53} - 1$ o $9.007.199.254.740.991$) se representa con precisión. Es un rango mayor que el de los números enteros en Java. Siempre que los resultados se mantengan dentro de este rango, las operaciones aritméticas con números enteros también serán precisas. Fuera de este rango, se producirán errores de redondeo. Por ejemplo, `Number.MAX_SAFE_INTEGER * 10` se evalúa como `90071992547409900`.



Nota

Si el rango de los enteros es insuficiente, puede utilizar «enteros grandes», que pueden tener un número arbitrario de dígitos. Hablaremos más tarde de ellos.

Al igual que con los números de coma flotante en cualquier lenguaje de programación, no se pueden evitar los errores de redondeo con valores fraccionarios. Por ejemplo, `0,1 + 0,2` se evalúa como `0,30000000000000004`, al igual que en Java, C++ o Python. Esto es inevitable, ya que los números decimales como 0,1, 0,2 y 0,3 no tienen representaciones binarias exactas. Si necesita realizar cálculos con euros y centavos, debe representar todas las cantidades como múltiplos enteros de un centavo.

Para convertir una cadena en un número, puede utilizar las funciones `parseFloat` o `parseInt`:

```
const notQuitePi = parseFloat('3.14') // El número 3.14
const evenLessPi = parseInt('3') // El entero 3
```

El método `toString` convierte un número de nuevo en una cadena:

```
const notQuitePiString = notQuitePi.toString() // La cadena «3,14»
const evenLessPiString = (3).toString() // La cadena «3»
```



Nota

JavaScript, al igual que C++ pero a diferencia de Java, tiene tanto funciones como métodos. Las funciones `parseFloat` y `parseInt` no son métodos, por lo que no se invocan con la notación de punto.



Nota

Como se ha visto en el fragmento de código anterior, se pueden aplicar métodos a literales numéricos. Sin embargo, hay que encerrar el literal numérico entre paréntesis para que el punto no se interprete como separador decimal.



Precaución

¿Qué ocurre cuando se utiliza un número fraccionario cuando se espera un entero? Depende de la situación. Supongamos que extraes una subcadena de una cadena. Entonces, las posiciones fraccionarias se truncan al siguiente entero más pequeño:

```
'Hello'.substring(0, 2.5) // La cadena 'He'
```

Pero cuando se proporciona un índice fraccionario, el resultado es indefinido:

```
'Hello'[2.5] // indefinido
```

No vale la pena averiguar cuándo un número fraccionario funciona como un entero. Si se encuentra en una situación así, deje clara su intención llamando a `Math.trunc(x)` para descartar la parte fraccionaria, o a `Math.round(x)` para redondear al entero más cercano.

Si divide por cero, el resultado es Infinito o -Infinito. Sin embargo, `0 / 0` es NaN, la constante «no es un número».

Algunas funciones que producen números devuelven NaN para indicar una entrada defectuosa. Por ejemplo, `parseFloat('pie')` es NaN.

1.7 Operadores aritméticos

JavaScript tiene los operadores habituales `+` `-` `*` `/` para la suma, la resta, la multiplicación y la división. Ten en cuenta que el operador `/` siempre da un resultado de punto flotante, incluso si ambos operandos son enteros. Por ejemplo, `1 / 2` es 0,5, no 0 como sería en Java o C++.

El operador `%` da como resultado el resto de la división entera para operandos enteros no negativos, al igual que en Java, C++ y Python. Por ejemplo, si `k` es un entero no negativo, entonces `k % 2` es 0 si `k` es par, y 1 si `k` es impar.

Si `k` y `n` son valores positivos, posiblemente fraccionarios, entonces `k % n` es el valor que se obtiene restando `n` de `k` hasta que el resultado sea menor que `n`. Por ejemplo, `3,5 % 1,2` es 1,1, el resultado de restar 1,2 dos veces.

El operador `**` denota «elevar a una potencia», al igual que en Python (y desde Fortran). El valor de `2 ** 10` es 1024, `2 ** -1` es 0,5 y `2 ** 0,5` es la raíz cuadrada de 2.

Si un operando de cualquier operador aritmético es el valor «no es un número» NaN, el resultado es nuevamente NaN.

Al igual que en Java, C++ y Python, se pueden combinar operaciones aritméticas y de asignación:

```
contador += 10 // Lo mismo que contador = contador + 10
```

Los operadores ++ y -- incrementan y decrementan una variable:

```
contador++ // Lo mismo que contador = contador + 1
```



Precaución

Al igual que Java y C++, JavaScript copia el lenguaje C, en el que ++ se puede aplicar antes o después de una variable, lo que da como resultado el valor de preincremento o posincremento.

```
let contador = 0
let acertijo = contador++
let enigma = ++contador
```

¿Cuáles son los valores de acertijo y enigma? Si no lo sabe, puede averiguarlo analizando cuidadosamente la descripción anterior, probándolo o recurriendo a la fuente de sabiduría que es Internet. Sin embargo, le recomiendo que nunca escriba código que dependa de este conocimiento.

Algunos programadores consideran que los operadores ++ y -- son tan reprobables que deciden no utilizarlos nunca. Y realmente no hay necesidad de hacerlo, ya que, al fin y al cabo, `counter += 1` no es mucho más largo que `counter++`. En este tema, utilizaré los operadores ++ y --, pero nunca en una situación en la que se capture su valor.

Al igual que en Java, el operador + también se utiliza para la concatenación de cadenas. Si `s` es una cadena y `x` un valor de cualquier tipo, entonces `s + x` y `x + s` son cadenas, obtenidas al convertir `x` en una cadena y unirla con `s`.

Por ejemplo,

```
let counter = 7
let agent = '00' + counter // La cadena '007'
```



Precaución

Como ha visto, la expresión `x + y` es un número si ambos operandos son números, y una cadena si al menos uno de los operandos es una cadena. En todos los demás casos, las reglas se vuelven complejas y los resultados rara vez son útiles. O bien ambos operandos se convierten en cadenas y se concatenan, o bien ambos se convierten en números y se suman. Por ejemplo, la expresión `null + undefined` se evalúa como la suma numérica `0 + NaN`, lo que da como resultado `NaN` (véase la **tabla 1-1**).

Con los demás operadores aritméticos, solo se intenta la conversión a números. Por ejemplo, el valor de `6 * '7'` es 42: la cadena '7' se convierte al número 7.

Tabla 1-1 Conversión a números y cadenas

Valor	A número	A cadena
Un número	El propio	Una cadena que contiene los dígitos del número

Valor	A número	A cadena
Una cadena que contiene los dígitos de un número	El valor numérico	En sí mismo
La cadena vacía ""	0	"
Cualquier otra cadena	NaN	La propia
false	0	«false»
verdadero	1	«verdadero»
nulo	0	«nulo»
indefinido	NaN	«indefinido»
La matriz vacía []	0	"
Una matriz que contiene un solo número	El número	Una cadena que contiene los dígitos del número
Otras matrices	NaN	Los elementos convertidos en cadenas y unidos por comas, como «1,2,3»
Objetos	Por defecto, NaN, pero se puede personalizar	Por defecto, «[object Object]», pero se puede personalizar



Consejo

No confíes en las conversiones automáticas de tipos con operadores aritméticos. Las reglas son confusas y pueden dar lugar a resultados no deseados. Si deseas procesar operandos que son cadenas o matrices de un solo elemento, conviértelos explícitamente.



Consejo

Prefiera los literales de plantilla a la concatenación de cadenas. De esta forma, no tendrá que recordar lo que hace el operador `+` con los operandos no numéricos.

1.8 Valores booleanos

El tipo booleano tiene dos valores, falso y verdadero. En una condición, los valores de cualquier tipo se convertirán a un valor booleano. Los valores 0, NaN, nulo, indefinido y la cadena vacía se convierten en falso, todos los demás en verdadero.

Esto parece bastante sencillo, pero, como verás en el siguiente capítulo, puede dar lugar a resultados muy confusos. Para minimizar la confusión, es buena idea utilizar valores booleanos reales para todas las condiciones.

1.9 null y undefined

JavaScript tiene dos formas de indicar la ausencia de un valor. Cuando se declara una variable, pero no se inicializa, su valor es indefinido. Esto suele ocurrir con las funciones. Cuando se llama a una función y no se proporciona un parámetro, la variable del parámetro tiene el valor indefinido.

El valor nulo tiene por objeto denotar la ausencia intencionada de un valor.

¿Es útil esta distinción? Hay dos escuelas de pensamiento. Algunos programadores piensan que tener dos valores «mínimos» es propenso a errores y sugieren que solo se utilice uno. En ese caso, se debe utilizar `undefined`. No se puede evitar `undefined` en el lenguaje JavaScript, pero se puede (en su mayor parte) evitar `null`.

El punto de vista opuesto es que nunca se deben establecer valores como indefinidos y nunca se debe devolver indefinido desde una función, sino utilizar siempre nulo para los valores que faltan. Entonces, indefinido puede indicar un problema grave.



Consejo

En cualquier proyecto, decídase explícitamente por uno u otro enfoque: utilice `undefined` o `null` para indicar la ausencia intencionada de un valor. De lo contrario, acabará en discusiones filosóficas sin sentido y comprobaciones innecesarias tanto de `undefined` como de `null`.



Precaución

A diferencia de `null`, `undefined` *no* es una palabra reservada. Es una variable en el ámbito global. ¡Antiguamente, se podía asignar un nuevo valor a la variable global `undefined`! Era claramente una idea terrible, y ahora `undefined` es una constante. Sin embargo, todavía se pueden declarar variables *locales* llamadas `undefined`. Por supuesto, eso también es una mala idea. Tampoco se deben declarar variables locales NaN e Infinity.

1.10 Literales de cadena

Los literales de cadena se escriben entre comillas simples o dobles: `'Hola'` o `"Hola"`. En este libro, siempre utilizo comillas simples como delimitadores.

Si utilizas una comilla dentro de una cadena delimitada por el mismo tipo de comilla, escápala con una barra invertida. También debes escapar las barras invertidas y los caracteres de control de la **Tabla 1-2**.

Por ejemplo, `'\\\"\\n'` es una cadena de longitud 5, que contiene `\"` seguido de una nueva línea.

Tabla 1-2 Secuencias de escape para caracteres especiales

Secuencia de escape	Nombre	Valor Unicode
<code>\b</code>	Retroceso	<code>\u{0008}</code>
<code>\t</code>	Tabulación	<code>\u{0009}</code>
<code>\n</code>	Salto de línea	<code>\u{000A}</code>
<code>\r</code>	Retorno de carro	<code>\u{000D}</code>
<code>\f</code>	Avance de página	<code>\u{000C}</code>
<code>\v</code>	Tabulación vertical	<code>\u{000B}</code>
<code>\'</code>	Comilla simple	<code>\u{0027}</code>
<code>\"</code>	Comilla doble	<code>\u{0022}</code>
<code>\\</code>	Barra invertida	<code>\u{005C}</code>
<code>\newline</code>	Continuación a la siguiente línea	Nada, no se añade ningún salto de línea: «Hel\ lo» es la cadena «Hello»

Para incluir caracteres Unicode arbitrarios en una cadena JavaScript, basta con escribirlos o pegarlos, siempre que el archivo fuente utilice una codificación adecuada (como UTF-8):

```
let greeting = 'Hello 🌐'
```

Si es importante mantener los archivos en ASCII, puede utilizar la notación `\u{punto de código}`:

```
let greeting = 'Hola \u{1F310}'
```

Desgraciadamente, hay un giro desagradable en Unicode en JavaScript. Para comprender los detalles, tenemos que profundizar en la historia de Unicode. Antes de Unicode, existía una mezcla de codificaciones de caracteres incompatibles en la que una secuencia de bytes podía significar cosas muy diferentes para los lectores de Estados Unidos, Rusia o China.

Unicode se diseñó para resolver estos problemas. Cuando se inició el esfuerzo de unificación en la década de 1980, se consideró que un código de 16 bits era más que suficiente para codificar todos los caracteres utilizados en todos los idiomas del mundo, con margen para una futura expansión. En 1991 se lanzó Unicode 1.0, que utilizaba algo menos de la mitad de los 65 536 valores de código disponibles en el espacio de caracteres universal (). Cuando se crearon JavaScript y Java en 1995, ambos adoptaron Unicode. En ambos lenguajes, las cadenas son secuencias de valores de 16 bits.

Por supuesto, con el tiempo, ocurrió lo inevitable. Unicode superó los 65 536 caracteres. Ahora, Unicode utiliza 21 bits, y todo el mundo cree que es realmente suficiente. Pero JavaScript se ha quedado estancado en los valores de 16 bits.

Necesitamos un poco de terminología para explicar cómo se resuelve este problema. Un *punto de código* Unicode es un valor de 21 bits asociado a un carácter. JavaScript utiliza la codificación UTF-16, que representa todos los puntos de código Unicode con uno o dos valores de 16 bits llamados *unidades de código*. Los caracteres hasta `\u{FFFF}` utilizan una unidad de código. Todos los demás caracteres se codifican con dos unidades de código, tomadas de un área reservada que no codifica ningún carácter. Por ejemplo, `\u{1F310}` se codifica como la secuencia `0xD83C 0xDF10`. (Consulte <http://en.wikipedia.org/wiki/UTF-16> para obtener una descripción del algoritmo de codificación).

No es necesario conocer los detalles de la codificación, pero sí hay que saber que algunos caracteres requieren una sola unidad de código de 16 bits y otros requieren dos.

Por ejemplo, la cadena «Hello 🌐» tiene una «longitud» de 8, aunque contiene siete caracteres Unicode. (Observe el espacio entre Hello y 🌐). Puede utilizar el operador de corchetes para acceder a las unidades de código de una cadena. La expresión `greeting[0]` es una cadena que consta de una sola letra «H». Sin embargo, el operador de corchetes no funciona con caracteres que requieren dos unidades de código. Las unidades de código del carácter «🌐» se encuentran en las posiciones 6 y 7. Las expresiones `greeting[6]` y `greeting[7]` son cadenas de longitud 1, cada una de las cuales contiene una sola unidad de código que no codifica un carácter. En otras palabras, no son cadenas Unicode adecuadas.



Nota

Puede proporcionar unidades de código de 16 bits en literales de cadena. En ese caso, omita las llaves: `\uD83C\uDF10`. Para unidades de código hasta `\u{0xFF}`, puede utilizar «escapes hexadecimales», por ejemplo, `\xA0` en lugar de `\u{00A0}`. No se me ocurre ninguna buena razón para hacer ninguna de las dos cosas.

1.11 Literales de plantilla

Los literales de plantilla son cadenas que pueden contener expresiones y abarcar varias líneas. Estas cadenas están delimitadas por comillas invertidas (``...``). Por ejemplo,

```
let destination = 'mundo' // Una cadena regular
let greeting = `Hola, ${destination.toUpperCase()}!` // Un literal de plantilla
```

Las expresiones incrustadas dentro de `${...}` se evalúan, se convierten en una cadena si es necesario y se insertan en la plantilla. En este caso, el resultado es la cadena

Hola, ¡MUNDO!

Puede anidar literales de plantilla dentro de las expresiones `${...}`:

```
saludo = `Hola, ${firstname.length > 0 ? `${firstname[0]}. ` : '' } ${lastname}`
```

Cualquier salto de línea dentro del literal de plantilla se incluye en la cadena. Por ejemplo,

```
saludo = `
```

establece el saludo en la cadena

```
'<div>Hola</div>\n<div>Mundo</div>\n'
```

con un salto de línea después de cada línea. (Los finales de línea de Windows \r\n en el archivo fuente se convierten a finales de línea de Unix \n en la cadena resultante).

Para incluir comillas invertidas, signos de dólar o barras invertidas en literales de plantilla, escápelos con barras invertidas: ``\\$`` es la cadena que contiene los tres caracteres `\\$`.



Nota

Una *plantilla literal etiquetada* es una plantilla literal precedida por una función, como esta:

```
html`<div>Hola, ${destination}</div>`
```

En este ejemplo, la función html se invoca con los fragmentos de plantilla

```
'<div>Hola, '
```

Y

```
'</div>'
```

y el valor de la expresión destino.

1.12 Objetos

Los objetos JavaScript son muy diferentes de los de los lenguajes basados en clases, como Java y C++. Un objeto JavaScript es simplemente un conjunto de pares nombre/valor o «propiedades», como este:

```
{ name: 'Harry Smith', age: 42 }
```

Este tipo de objeto solo tiene datos públicos y no tiene encapsulación ni comportamiento. El objeto no es una instancia de ninguna clase en particular. En otras palabras, no se parece en nada a un objeto de la programación orientada a objetos tradicional. Es posible declarar clases y métodos, pero los mecanismos son muy diferentes a los de la mayoría de los demás lenguajes.

Por supuesto, puede almacenar un objeto en una variable:

```
const harry = { name: 'Harry Smith', age: 42 }
```

Una vez que tienes una variable de este tipo, puedes acceder a las propiedades del objeto con la notación habitual de puntos:

```
let harrysAge = harry.age
```

Puedes modificar las propiedades existentes o añadir nuevas propiedades:

```
harry.age = 40
harry.salary = 90000
```



Nota

La variable `harry` se declaró como `const`, pero como acaba de ver, puede mutar el objeto al que hace referencia. Sin embargo, no puede asignar un valor diferente a una variable `const`.

```
const sally = { name: 'Sally Lee' }  
sally.age = 28 // OK: modifica el objeto al que hace referencia sally  
sally = { name: 'Sally Albright' }  
// Error: no se puede asignar un valor diferente a una variable const
```

En otras palabras, `const` es como `final` en Java y no se parece en nada a `const` en C++.

Utilice el operador `delete` para eliminar una propiedad:

```
delete harry.salary
```

Al acceder a una propiedad inexistente se obtiene un resultado indefinido:

```
let boss = harry.supervisor // undefined
```

Se puede calcular el nombre de una propiedad. A continuación, utilice corchetes de matriz para acceder al valor de la propiedad:

```
let field = 'Age'  
let harrysAge = harry[field.toLowerCase()]
```

1.13 Sintaxis de objetos literales

Un objeto literal puede tener una coma final. Esto facilita añadir otras propiedades a medida que el código evoluciona:

```
let harry = {  
  name: 'Harry Smith',  
  age: 42, // Añade más propiedades a continuación  
}
```

Muy a menudo, al declarar un objeto literal, los valores de las propiedades se almacenan en variables cuyos nombres son iguales a los nombres de las propiedades. Por ejemplo,

```
let age = 43  
let harry = { name: 'Harry Smith', age: age }  
// La propiedad « age » se establece en el valor de la variable age
```

Hay un atajo para esta situación:

```
let harry = { name: 'Harry Smith', age } // La propiedad age ahora es 43
```

Utilice corchetes para los nombres de propiedades calculadas en literales de objeto:

```
let harry = { name: 'Harry Smith', [field.toLowerCase()] : 42 }
```

El nombre de una propiedad siempre es una cadena. Si el nombre no sigue las reglas de un identificador, póngalo entre comillas en un literal de objeto:

```
let harry = { name: 'Harry Smith', 'favorite beer': 'IPA' }
```

Para acceder a dicha propiedad, no puede utilizar la notación de punto. Utilice corchetes en su lugar:

```
harry['favorite beer'] = 'Lager'
```

Estos nombres de propiedades no son habituales, pero a veces pueden resultar útiles. Por ejemplo, puede tener un objeto cuyos nombres de propiedades sean nombres de archivos y cuyos valores de propiedad sean el contenido de esos archivos.



Precaución

Hay situaciones de análisis en las que una llave de apertura puede indicar un objeto literal o una instrucción de bloque. En esos casos, la instrucción de bloque tiene prioridad. Por ejemplo, si escribes

```
{ } - 1
```

en la consola del navegador o en Node.js, se ejecuta el bloque vacío. A continuación, se evalúa y se muestra la expresión `- 1`.

Por el contrario, en la expresión

```
1 - { }
```

`{ }` es un objeto vacío que se convierte en NaN. A continuación, se muestra el resultado (también NaN).

Esta ambigüedad no suele darse en programas prácticos. Cuando se forma un objeto literal, normalmente se almacena en una variable, se pasa como argumento o se devuelve como resultado. En todas esas situaciones, el analizador no esperaría un bloque.

Si alguna vez se encuentra con una situación en la que un literal de objeto se analiza erróneamente como un bloque, la solución es sencilla: encierre el literal de objeto entre paréntesis.

1.14 Matrices

En JavaScript, una matriz es simplemente un objeto cuyos nombres de propiedad son las cadenas «0», «1», «2», etc. (Se utilizan cadenas porque los números no pueden ser nombres de propiedad).

Puedes declarar literales de matriz encerrando sus elementos entre corchetes:

```
const numbers = [1, 2, 3, 'many']
```

Este es un objeto con cinco propiedades: «0», «1», «2», «3» y «length».

La propiedad `length` es uno más que el índice más alto, convertido a un número. El valor de `numbers.length` es el número 4.

Debe utilizar la notación entre corchetes para acceder a las cuatro primeras propiedades: `numbers['1']` es 2. Para su comodidad, el argumento dentro de los corchetes se convierte automáticamente en una cadena. Puede utilizar `numbers[1]` en su lugar, lo que le da la impresión de estar trabajando con una matriz en un lenguaje como Java o C++.

Tenga en cuenta que los tipos de elementos de una matriz no tienen por qué coincidir. La matriz `numbers` contiene tres números y una cadena.

Una matriz puede tener elementos que faltan:

```
const someNumbers = [ , 2, , 9] // No hay propiedades '0', '2'
```

Al igual que con cualquier objeto, una propiedad inexistente tiene el valor indefinido. Por ejemplo, `someNumbers[0]` y `someNumbers[6]` son indefinidos.

Puede añadir nuevos elementos al final:

```
someNumbers[6] = 11 // Ahora someNumbers tiene una longitud de 7
```

Ten en cuenta que, al igual que con todos los objetos, puedes cambiar las propiedades de una matriz a la que hace referencia una variable `const`.



Nota

Una coma *final* no indica que falte un elemento. Por ejemplo, `[1, 2, 7, 9,]` tiene cuatro elementos y el índice más alto es 3. Al igual que con los literales de objetos, las comas finales están pensadas para literales que pueden ampliarse con el tiempo, como, por ejemplo:

```
const developers = [  
  'Harry Smith',  
  'Sally Lee',  
  // Añadir más elementos arriba  
]
```

Dado que las matrices son objetos, puedes añadir propiedades arbitrarias:

```
numbers.lucky = true
```

Esto no es habitual, pero es perfectamente válido en JavaScript.

El operador `typeof` devuelve «objeto» para una matriz. Para comprobar si un objeto es una matriz, llama a `Array.isArray(obj)`.

Cuando es necesario convertir una matriz en una cadena, todos los elementos se convierten en cadenas y se unen con comas. Por ejemplo,

```
'' + [1, 2, 3]
```

es la cadena «1,2,3».

Una matriz de longitud 0 se convierte en una cadena vacía.

JavaScript, al igual que Java, no tiene nociones de matrices multidimensionales, pero se pueden simular con matrices de matrices. Por ejemplo,

```
const melancholyMagicSquare = [  
  [16, 3, 2, 13],  
  [5, 10, 11, 8],  
  [9, 6, 7, 12],  
  [4, 15, 14, 1]  
]
```

A continuación, utilice dos pares de corchetes para acceder a un elemento:

```
melancholyMagicSquare[1][2] // 11
```

1.15 JSON

La notación de objetos JavaScript o JSON es un formato de texto ligero para intercambiar datos de objetos entre aplicaciones (que pueden estar implementadas o no en JavaScript).

En pocas palabras, JSON utiliza la sintaxis de JavaScript para los literales de objetos y matrices, con algunas restricciones:

Los valores son literales de objetos, literales de matrices, cadenas, números de punto flotante y los valores `true`, `false` y `null`.

- Todas las cadenas están delimitadas por comillas dobles, no por comillas simples.
- Todos los nombres de propiedades están delimitados por comillas dobles.
- No hay comas finales ni elementos omitidos.

Consulte www.json.org para obtener una descripción formal de la notación.

Un ejemplo de una cadena JSON es:

```
{ "name": "Harry Smith", "age": 42, "lucky numbers": [17, 29], "lucky": false }
```

El método `JSON.stringify` convierte un objeto JavaScript en una cadena JSON, y `JSON.parse` analiza una cadena JSON, generando un objeto JavaScript. Estos métodos se utilizan habitualmente cuando se comunica con un servidor a través de HTTP.



Precaución

`JSON.stringify` omite las propiedades del objeto cuyo valor es indefinido y convierte los elementos de la matriz con valores indefinidos en nulos. Por ejemplo, `JSON.stringify({ name: ['Harry', undefined, 'Smith'], age: undefined })` es la cadena `'{"name":["Harry",null,"Smith"]}'`.

Algunos programadores utilizan el método `JSON.stringify` para el registro. Un comando de registro

```
console.log(`harry=${harry}`)
```

te da un mensaje inútil

```
harry=[object Object]
```

Una solución es llamar a `JSON.stringify`:

```
console.log(`harry=${JSON.stringify(harry)}`)
```

Ten en cuenta que este problema solo se produce con cadenas que contienen objetos. Si registras un objeto por sí solo, la consola lo muestra correctamente. Una alternativa sencilla es registrar los nombres y los valores por separado:

```
console.log('harry=', harry, 'sally=', sally)
```

O, aún más fácil, colócalos en un objeto:

```
console.log({harry, sally}) // Registra el objeto { harry: { . . . }, sally: { . . . } }
```

1.16 Desestructuración

La desestructuración es una sintaxis conveniente para obtener los elementos de una matriz o los valores de un objeto. Al igual que con los otros temas de nivel intermedio de este capítulo, no dude en saltarse esta sección hasta que esté listo para ella.

En esta sección, comenzamos con la sintaxis básica. La siguiente sección cubre algunos de los puntos más delicados.

Veamos primero las matrices. Supongamos que tienes una matriz `pair` con dos elementos. Por supuesto, puedes obtener los elementos de esta manera:

```
let first = pair[0]
let second = pair[1]
```

Con la desestructuración, esto se convierte en:

```
let [first, second] = pair
```

Esta instrucción declara las variables `first` y `second` y las inicializa con `pair[0]` y `pair[1]`.

El lado izquierdo de una asignación de desestructuración no es en realidad un literal de matriz. Después de todo, `first` y `second` aún no existen. Piensa en el lado izquierdo como un patrón que describe cómo deben coincidir las variables con el lado derecho.

Considera este caso más complejo y observa cómo se emparejan las variables con los elementos de la matriz:

```
let [first, [second, third]] = [1, [2, 3]]
// Establece first en 1, second en 2 y third en 3
```

La matriz de la parte derecha puede ser más larga que el patrón de la parte izquierda. Los elementos no emparejados simplemente se ignoran:

```
let [primero, segundo] = [1, 2, 3]
```

Si la matriz es más corta, las variables no coincidentes se establecen como indefinidas:

```
let [primero, segundo] = [1]
// Establece first en 1 y second en indefinido
```

Si las variables `first` y `second` ya están declaradas, puede utilizar la desestructuración para establecerles nuevos valores:

```
[first, second] = [4, 5]
```



Consejo

Para intercambiar los valores de las variables `x` e `y`, simplemente utilice:

```
[x, y] = [y, x]
```

Si utiliza la desestructuración para una asignación, el lado izquierdo no tiene por qué estar formado por variables. Puede utilizar cualquier `lvalue`, es decir, expresiones que puedan aparecer en el lado izquierdo de una asignación. Por ejemplo, esta es una desestructuración válida:

```
[numbers[0], harry.age] = [13, 42] // Igual que numbers[0] = 13; harry.age = 42
```

La desestructuración para objetos es similar. Utiliza nombres de propiedades en lugar de posiciones de matriz:

```
let harry = { name: 'Harry', age: 42 }

let { name: harrysName, age: harrysAge } = harry
```

Este fragmento de código declara dos variables `harrysName` y `harrysAge` y las inicializa con los valores de las propiedades `name` y `age` del objeto de la derecha.

Tenga en cuenta que el lado izquierdo *no* es un objeto literal. Es un patrón para mostrar cómo se emparejan las variables con el lado derecho.

La desestructuración con objetos es más convincente cuando la propiedad tiene el mismo nombre que la variable. En ese caso, se puede omitir el nombre de la propiedad y los dos puntos. Esta instrucción declara dos variables, `name` y `age`, y las inicializa con las propiedades del objeto de la derecha que tienen el mismo nombre:

```
let { name, age } = harry
```

Es lo mismo que:

```
let { name: name, age: age } = harry
```

o, por supuesto,

```
let name = harry.name  
let age = harry.age
```



Precaución

Si utiliza la desestructuración de objetos para establecer variables existentes, debe encerrar la expresión de asignación entre paréntesis:

```
({name, age} = sally)
```

De lo contrario, la llave de apertura se analizará como el inicio de una instrucción de bloque.

1.17 Desestructuración avanzada

En la sección anterior, nos centramos en las partes más fáciles y atractivas de la sintaxis de desestructuración. En esta sección avanzada, verás características adicionales que son potentes, pero menos intuitivas. No dudes en saltarte esta sección y volver a ella cuando te sientas cómodo con los conceptos básicos.

1.17.1 Más información sobre la desestructuración de objetos

Puedes desestructurar objetos anidados:

```
let pat = { name: 'Pat', birthday: { day: 14, month: 3, year: 2000 } }  
let { birthday: { year: patsBirthYear } } = pat  
// Declara la variable patsBirthYear y la inicializa en 2000
```

Una vez más, tenga en cuenta que el lado izquierdo de la segunda instrucción *no es un objeto*. Es un patrón para hacer coincidir las variables con el lado derecho. La instrucción tiene el mismo efecto que:

```
let patsBirthYear = pat.birthday.year
```

Al igual que con los literales de objeto, se admiten nombres de propiedades calculadas:

```
let field = 'Age'  
let { [field.toLowerCase()]: harrysAge } = harry  
// Establece el valor en harry[field.toLowerCase()]
```

1.17.2 Declaraciones de resto

Al desestructurar una matriz, puede capturar cualquier elemento restante en una matriz. Añada un prefijo `...` antes del nombre de la variable.

```
números = [1, 7, 2, 9]
let [first, second, ...others] = números
// first es 1, second es 7 y others es [2, 9]
```

Si la matriz de la derecha no tiene suficientes elementos, la variable `rest` se convierte en una matriz vacía:

```
let [first, second, ...others] = [42]

// first es 42, second es indefinido y others es []
```

Una declaración *rest* también funciona para objetos:

```
let { name, ...allButName } = harry
// allButName es { edad: 42 }
```

La variable `allButName` se establece en un objeto que contiene todas las propiedades excepto la que tiene la clave `name`.

1.17.3 Valores predeterminados

Para cada variable, puede proporcionar un valor predeterminado que se utilizará si el valor deseado no está presente en el objeto o la matriz, o si el valor no está definido. Coloque `=` y una expresión después del nombre de la variable:

```
let [first, second = 0] = [42]
// Establece first en 42 y second en 0, ya que el lado derecho no tiene
// ningún elemento coincidente
let { nickname = 'None' } = harry
// Establece nickname en 'None', ya que harry no tiene la propiedad nickname
```

Las expresiones predeterminadas pueden utilizar variables establecidas previamente:

```
let { name, nickname = name } = harry
// Tanto el name como el nickname se establecen en harry.name
```

Esta es una aplicación típica de la desestructuración con valores predeterminados. Supongamos que se le proporciona un objeto que describe ciertos detalles de procesamiento, por ejemplo, instrucciones de formato. Si no se proporciona una propiedad concreta, entonces se desea utilizar un valor predeterminado:

```
let config = { separator: ';' }
const { separator = ',', leftDelimiter = '[', rightDelimiter = ']' } = config
```

En el ejemplo, la variable `separator` se inicializa con el separador personalizado y se utilizan los delimitadores predeterminados porque no se proporcionan en la configuración. La sintaxis de desestructuración es bastante más concisa que buscar cada propiedad, comprobar si está definida y proporcionar el valor predeterminado si no lo está.